

ARTI 404 - Image Processing

Lab 4: Intensity Transformations and Filtering (Spatial Domain)

Student 2240009025 Nawaf Rabea Shahbal

Task 1: Thresholding

Thresholding turns a grayscale image into a binary image. Pixels above the threshold become white (255) and pixels below it become black (0). I will try 5 different threshold values and compare them.

```
In [1]: import cv2
import os
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, img_as_float, exposure, io
```

I don't have Parrot.png so I used the `camera` image from `skimage` and saved it in an `images` folder. Then I load it in grayscale with OpenCV like the manual says.

```
In [2]: os.makedirs('images', exist_ok=True)
io.imsave('images/Parrot.png', data.camera(), check_contrast=False)

# Load the image in grayscale
img = cv2.imread('images/Parrot.png', cv2.IMREAD_GRAYSCALE)
print(img.shape, img.dtype)
```

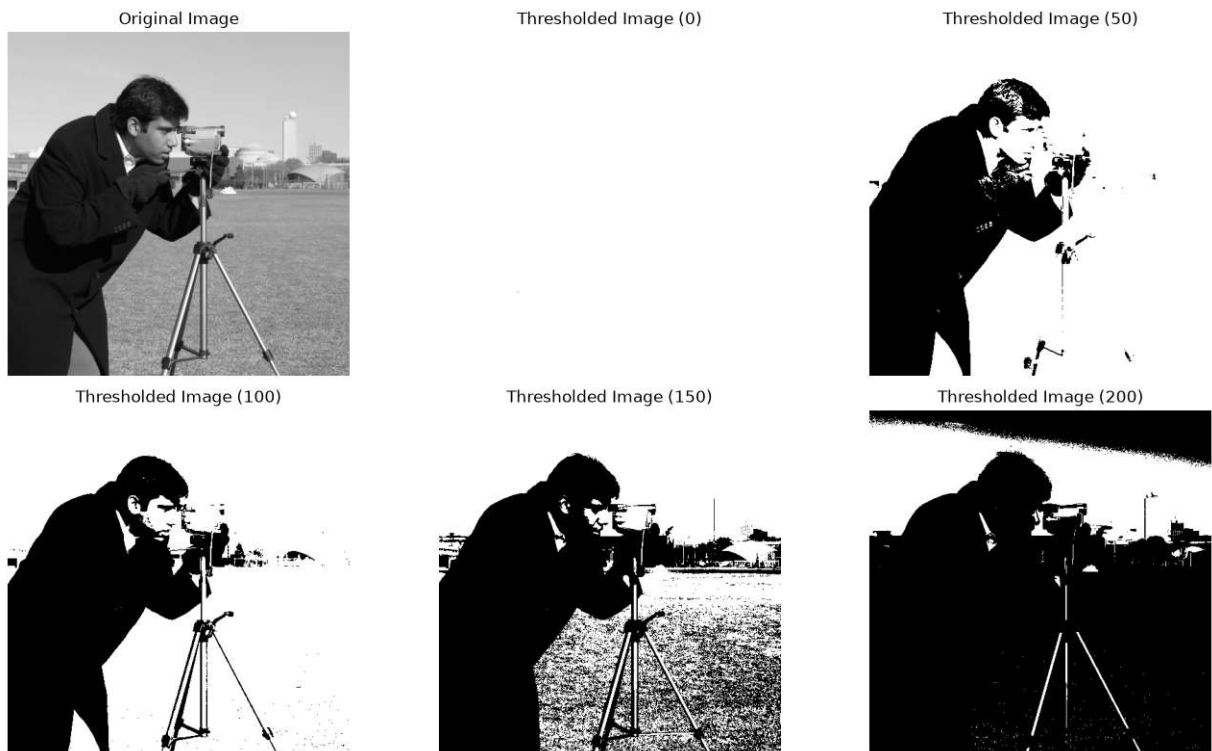
(512, 512) uint8

```
In [3]: # Set the threshold values
threshold_values = [0, 50, 100, 150, 200]

plt.figure(figsize=(14, 8))
plt.subplot(2, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

# Apply thresholding with different threshold values
for i, threshold_value in enumerate(threshold_values):
    ret, thresh = cv2.threshold(img, threshold_value, 255, cv2.THRESH_BINARY)
    plt.subplot(2, 3, i + 2)
    plt.imshow(thresh, cmap='gray')
    plt.title(f'Thresholded Image ({threshold_value})')
    plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```



Observation: With threshold 0 almost everything is white because every pixel is above 0. When the threshold goes up, more pixels become black and we lose details. At 100-150 the man and the camera shape are still clear, but at 200 only the brightest areas (like the sky) are left. So the threshold should be chosen carefully depending on the image.

Task 2: Histogram Processing

Step 1: Load libraries

```
In [4]: import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from skimage import data, img_as_float
from skimage import exposure
```

Step 2: Load moon image

```
In [5]: img = data.moon()
print(img.shape, img.dtype, img.min(), img.max())

(512, 512) uint8 0 255
```

Step 3: Contrast stretching (2nd and 98th percentiles)

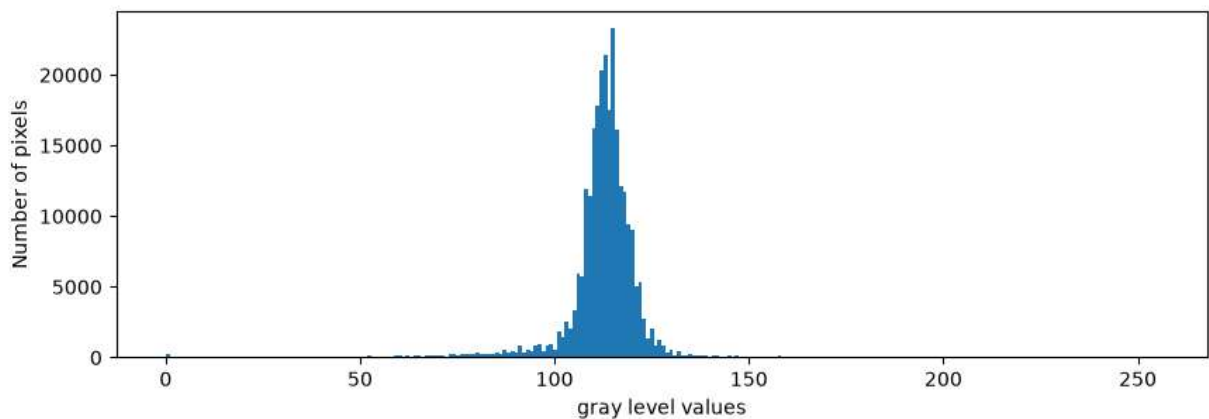
```
In [6]: # Contrast stretching
p2, p98 = np.percentile(img, (2, 98))
img_rescale = exposure.rescale_intensity(img, in_range=(p2, p98))
print(p2, p98)
```

78.0 129.0

Step 4: Display the images with their histograms

```
In [7]: fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img, cmap='gray')
plt.axis('off')
plt.title('Original image')
fig.add_subplot(2, 1, 2)
plt.hist(img.flat, bins=256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

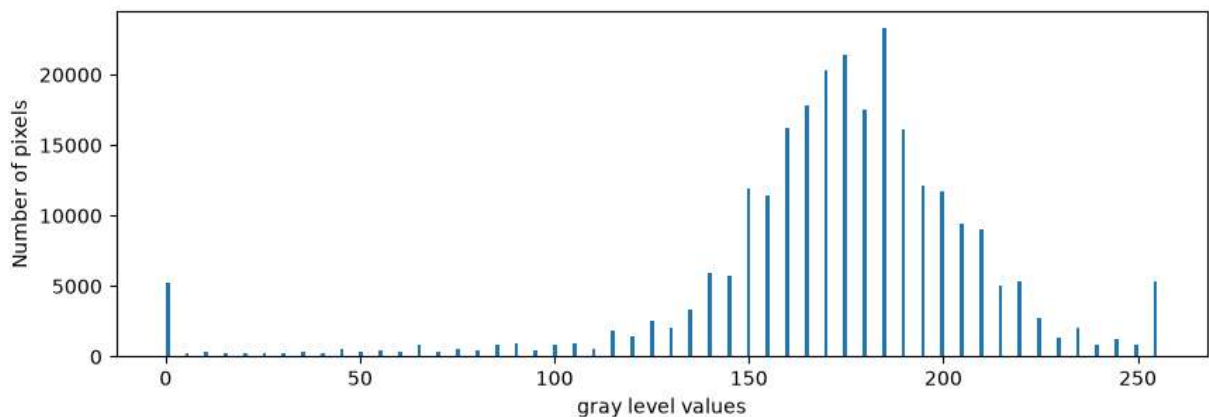
Original image



```
In [8]: fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap='gray')
plt.axis('off')
plt.title('Contrast stretch')
fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins=256, range=(0, 255))
plt.xlabel('gray level values')
```

```
plt.ylabel('Number of pixels')
plt.show()
```

Contrast stretch



Observation: The original moon image has most of its pixels in a narrow range of dark gray levels, so it looks dull. After contrast stretching the histogram is spread over the whole range 0-255 and the image has better contrast, the craters are easier to see.

Assessment

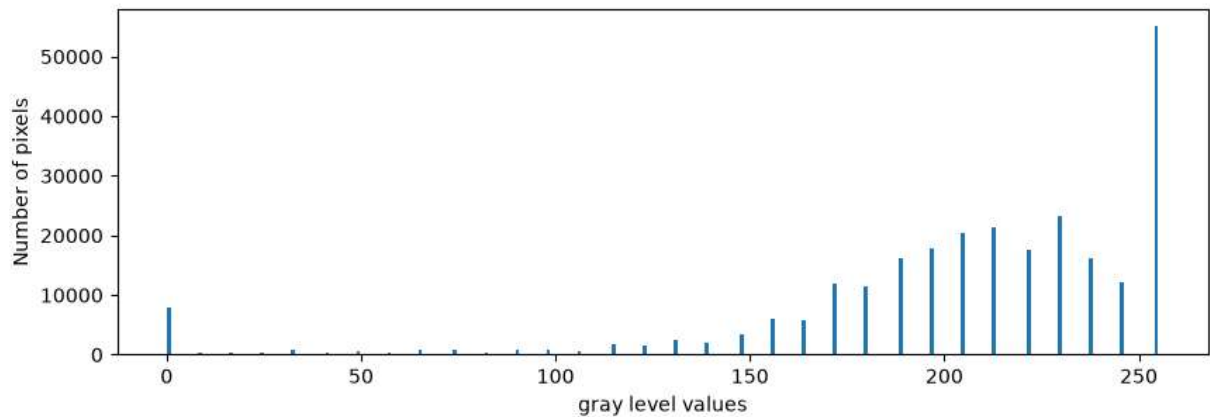
Task 1: Rescale intensity using the 3rd and 80th percentiles

```
In [9]: p3, p80 = np.percentile(img, (3, 80))
img_rescale2 = exposure.rescale_intensity(img, in_range=(p3, p80))
print(p3, p80)

fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale2, cmap='gray')
plt.axis('off')
plt.title('Contrast stretch (3rd - 80th percentile)')
fig.add_subplot(2, 1, 2)
plt.hist(img_rescale2.flat, bins=256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

87.0 118.0

Contrast stretch (3rd - 80th percentile)



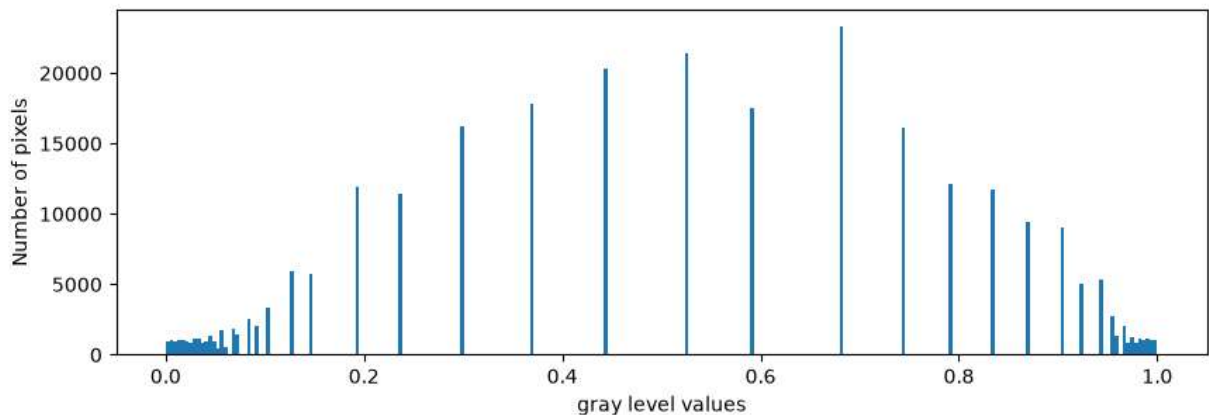
Observation: Using the 80th percentile as the upper limit makes more of the bright pixels saturate to 255, so the image becomes brighter and there is a big spike at white in the histogram.

Task 2: Histogram equalization (`exposure.equalize_hist`)

```
In [10]: img_eq = exposure.equalize_hist(img)  # output is float in range [0, 1]

fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img_eq, cmap='gray')
plt.axis('off')
plt.title('Histogram equalization')
fig.add_subplot(2, 1, 2)
plt.hist(img_eq.flat, bins=256, range=(0, 1))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

Histogram equalization



Observation: `equalize_hist` returns a float image between 0 and 1, so the histogram range is (0, 1). The histogram is now flatter (more uniform), The image has strong contrast.

Task 3: Histogram matching (rocket as reference, Chelsea as the image)

```
In [11]: reference = data.rocket()
image = data.chelsea()
print(reference.shape, image.shape)

# channel_axis=-1 because the images are RGB
matched = exposure.match_histograms(image, reference, channel_axis=-1)

fig, (ax1, ax2, ax3) = plt.subplots(nrows=1, ncols=3, figsize=(12, 4))
for aa in (ax1, ax2, ax3):
    aa.set_axis_off()

ax1.imshow(image)
ax1.set_title('Source (Chelsea)')
ax2.imshow(reference)
ax2.set_title('Reference (Rocket)')
ax3.imshow(matched)
ax3.set_title('Matched')
plt.tight_layout()
plt.show()
```

(427, 640, 3) (300, 451, 3)

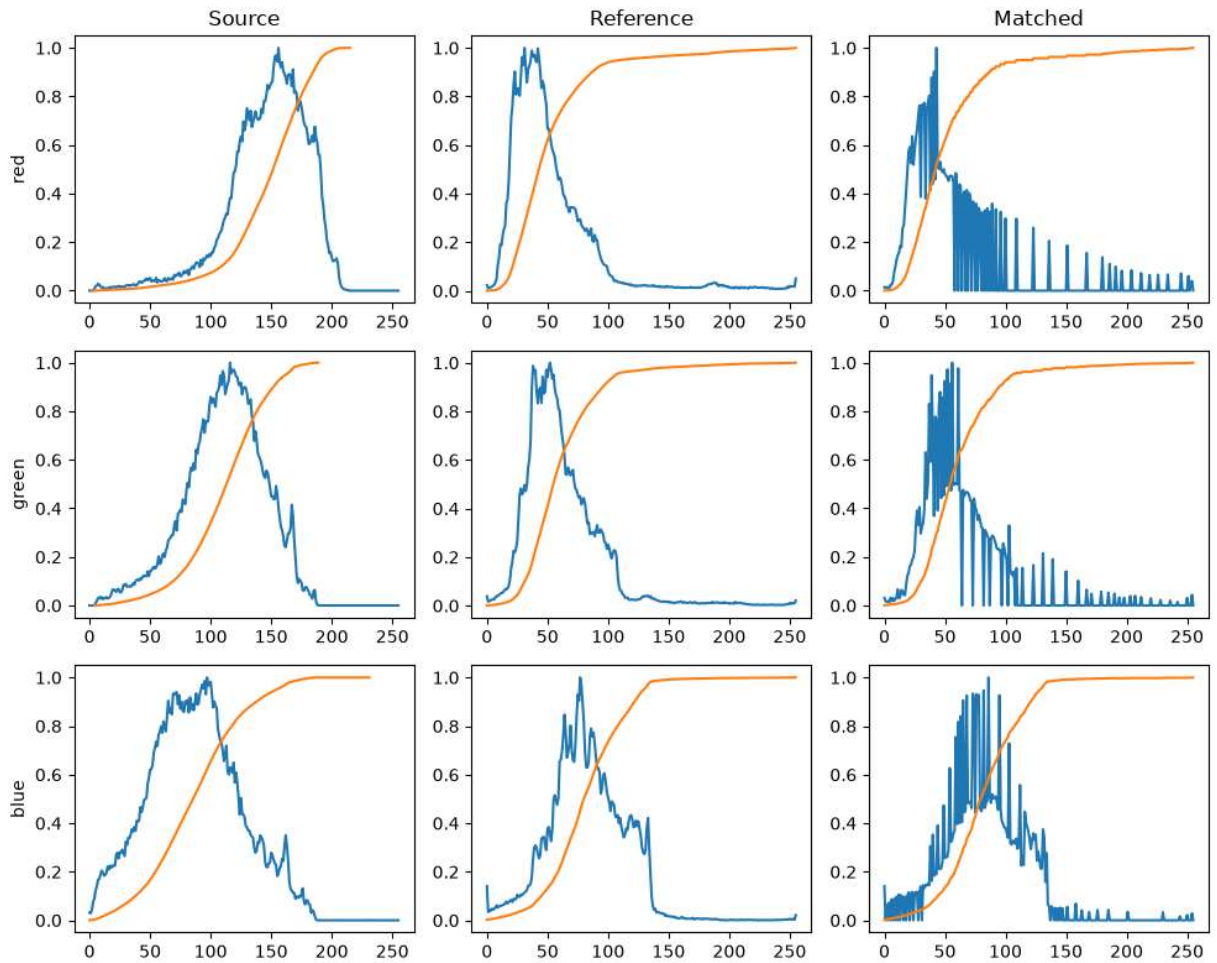


Now I plot the histogram of each channel (R, G, B) for the three images to check the matching.

```
In [12]: fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(10, 8))

for i, img_ in enumerate((image, reference, matched)):
    for c, c_color in enumerate(('red', 'green', 'blue')):
        img_hist, bins = exposure.histogram(img_[..., c], source_range='dtype')
        axes[c, i].plot(bins, img_hist / img_hist.max())
        img_cdf, bins = exposure.cumulative_distribution(img_[..., c])
        axes[c, i].plot(bins, img_cdf)
        axes[c, 0].set_ylabel(c_color)

axes[0, 0].set_title('Source')
axes[0, 1].set_title('Reference')
axes[0, 2].set_title('Matched')
plt.tight_layout()
plt.show()
```



Observation: After matching, the cat image takes the color distribution of the rocket image.

Conclusion

In this lab I used thresholding, contrast stretching, histogram equalization and histogram matching.